

G. Performance Analysis

Problem ID: Compiler

Time Limit: 1s

Memory Limit: 512MiB

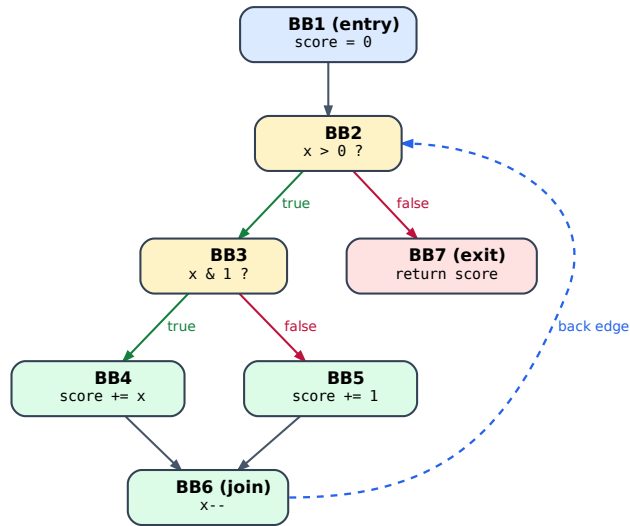


Figure 1: 一個含迴圈與條件分支的 CFG。每個框是一個 basic block； BB_6 是兩條分支的 join point， $BB_6 \rightarrow BB_2$ 是迴圈的 back edge。

tobiichi3227 跟 yushiuan9499 最近在開發 NCC，tobiichi3227 發現它程式執行太慢了，所以想給 NCC 加上一個功能，統計每一條 control-flow edge 會執行幾次。

編譯器將程式用控制流圖 (Control Flow Graph，後面簡稱 CFG) 表示，CFG 上面會有 N 個基本塊 (Basic Block，後面簡稱 BB)，每個 BB 會有幾個有向邊連到其他 BB，代表從 BB_i 可以走到 BB_j ，總共有 M 條有向邊。

程式執行會從入口 1 開始，最後從出口 N 離開結束執行，總共執行 R 次程式。

對於每條邊，編譯器在它所連接的 BB 上插入性能計數器 (BB_i 或 BB_j 都可以，編譯器只會選一個 BB 插入)，因為測量執行次數會造成額外的運算，所以插入每個計數器需要成本 c_e ，執行結束後就能知道該邊通過的次數。

對於沒有裝計數器的邊，可以用 flow conservation 推導計算得知：

- 對於普通節點: $\sum_{e \in \text{in}(v)} f_e = \sum_{e \in \text{out}(v)} f_e \quad (v \neq 1, N)$
- 對於入口: $\sum_{e \in \text{out}(1)} f_e - \sum_{e \in \text{in}(v)} f_e = R$
- 對於出口: $\sum_{e \in \text{in}(N)} f_e - \sum_{e \in \text{out}(N)} f_e = R$

求最小安裝成本，使得執行結束後能夠唯一確定每一條邊通過的次數。

— 輸入 —

第一行會有三個整數 N, M, R ，代表 BB 的數量、邊的數量與執行次數

接下來 M 行會有三個整數 u, v, C ，代表 BB_u 可以走到 BB_v ，且安裝計數器成本為 C

— 輸出 —

最小安裝成本

— 輸入限制 —

- $1 \leq N \leq 10^5$
- $1 \leq M \leq 2 \times 10^5$
- $2M \leq R \leq 10^{18}$
- $1 \leq u, v \leq N$
- $1 \leq C_e \leq 10^9$
- 允許 self-loop 與重邊
- 保證所有 BB 一定有一個 1 到 N 的路徑通過

— 子任務 —

編號	分數	額外限制
1	0	範例輸入輸出
2	5	$M = N - 1$, 且第 i 條邊為 $i \rightarrow i + 1$
3	10	$C_e = 1$
4	15	$M \leq 20$
5	15	$M = N$ 且無 self-loop 或重複連接
6	20	每個點雙連通分量皆為單邊分量，或其中每個頂點的分量內度數皆為 2
7	35	無額外限制

— 範例輸入 —

4 4 8
1 2 5
1 3 2
2 4 4
3 4 6

— 範例輸出 —

2

— 範例解釋 —

輸入的 CFG 的結構如下：

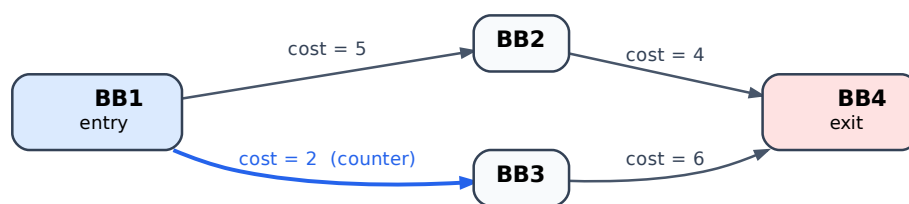


Figure 2: 範例一的 CFG。邊上標示安裝計數器的成本；藍色粗邊表示在 $1 \rightarrow 3$ 安裝計數器。

程式總共執行 8 次。每次執行會選擇下列其中一條路線：

$1 \rightarrow 2 \rightarrow 4$ 或 $1 \rightarrow 3 \rightarrow 4$

可以花費 2，在邊 $1 \rightarrow 3$ 上安裝計數器。

假設程式執行完畢後，計數器顯示邊 $1 \rightarrow 3$ 總共被通過 3 次。因為程式總共執行 8 次，所以邊 $1 \rightarrow 2$ 必定被通過 $8 - 3 = 5$ 次。

每次進入 BB_2 後都會沿著邊 $2 \rightarrow 4$ 離開，因此邊 $2 \rightarrow 4$ 也被通過 5 次。

同理，邊 $3 \rightarrow 4$ 被通過 3 次。

因此，只要在邊 $1 \rightarrow 3$ 上安裝一個計數器，就能確定所有邊的通過次數。

若不安裝任何計數器，則無法判斷 8 次執行中分別有多少次經過 BB_2 或 BB_3 。因此至少需要安裝一個計數器。

所有邊中最低的安裝成本為 2，所以最小總費用為 2。

— 範例輸入 —

4 7 14
1 2 8
2 4 5
1 3 7

3 4 4

2 3 6

2 2 3

1 2 9

— 範例輸出 —

21

— 範例輸入 —

6 6 12

1 2 9

2 3 8

3 4 3

2 4 7

4 5 6

5 6 5

— 範例輸出 —

3

— 範例輸入 —

8 9 18

1 2 10

2 3 8

3 5 2

2 4 6

4 5 5

5 6 7

6 7 4

7 8 3

6 8 9

— 範例輸出 —

5

－ 編譯器小教室：從 C Code 到 CFG －

C 的 if、while、for 等語法適合人類閱讀，但編譯器進行最佳化時，更希望控制流只有少數幾種明確操作：

```
goto L
if condition goto L_true else goto L_false
return value
```

把結構化控制流展開成 label 與 jump 的過程，通常稱為 **lowering**。下文使用的 goto-style 是為了教學設計的 pseudo IR：它和 LLVM IR 一樣用 label 與明確的 branch 描述控制流，但它**不是合法的 LLVM IR**。真正的 LLVM IR 有型別、SSA、phi 等規則，而且使用的 terminator 名稱是 br、ret、switch 等。

概念上的對應關係如下：

```
goto L
    <=> br label %L

if c goto T else goto F
    <=> br i1 %c, label %T, label %F

return x
    <=> ret i32 %x
```

把大括號中的 if、迴圈等結構展開成 label 與 goto，概念上正是 TOJ 756 練習的工作。即使考場沒有網路，也不影響閱讀以下內容；連結只供課後延伸。

Basic Block (BB)

Basic block 是一段**最大且連續**的指令序列，具有以下性質：

- 控制流只會從第一條指令進入，不會跳進 block 中間；
- 除了最後一條指令以外，中途不會跳到別處；
- block 內的指令一旦開始，就會照順序全部執行到 terminator。

在 LLVM-like IR 中，每個 BB 以 label 開始，以一條 terminator 結束。terminator 決定下一個 BB，或直接結束函式。

將一串 goto-style 指令切成 BB 時，可以先找出 **leaders**：

1. 函式的第一條指令是 leader；
2. 任何 jump 的目標 label 是 leader；
3. conditional branch、goto、return 後面的下一條指令若存在，也是新 block 的 leader。

每個 leader 到下一個 leader 之前的所有指令，會形成一個 BB。實作時通常會再刪除 unreachable blocks，並合併不必要的空 blocks。

從 BB 建立 CFG

CFG 中每個節點是一個 BB。若執行完 BB_u 後可能接著執行 BB_v ，就在圖上加入有向邊 $u \rightarrow v$ ：

- goto BB2：一條邊連到 BB_2 ；
- if c goto BB2 else BB3：分別連到 BB_2 與 BB_3 ；
- return：沒有函式內的 successor；
- 若 IR 允許 fall-through，則連到文字順序的下一個 BB。本文會把 jump 全部寫出來，避免隱藏的 fall-through。

因此，goto-style 的 label 先決定 BB 邊界，terminator 再決定 CFG edges。

常見 C 控制結構如何 Lower ?

以下例子都先改寫成 goto-style，再從每個 terminator 直接讀出 CFG。

1. 直線程式

```
x = a + b;  
y = x * 2;  
return y;
```

沒有 branch，整段就是一個 BB：

```
BB1:  
    x = a + b  
    y = x * 2  
    return y
```

return 是 terminator，因此不能再把其他可執行指令接在它後面。

2. if / else

```
if (x < 0)  
    y = -x;  
else  
    y = x;  
use(y);
```

Lowering 後：

```
BB1: if x < 0 goto BB2 else goto BB3  
BB2: y = -x; goto BB4  
BB3: y = x;  goto BB4  
BB4: use(y); ...
```

CFG edges 為 $1 \rightarrow 2$ 、 $1 \rightarrow 3$ 、 $2 \rightarrow 4$ 、 $3 \rightarrow 4$ 。BB₄ 是兩條分支重新會合的 **join point**。若原始程式沒有 else，則 BB₁ 的 false edge 可以直接連到 join point。

3. while

```
while (x > 0) {  
    sum += x;  
    --x;  
}  
use(sum);
```

Lowering 後：

```
BB1:  
    goto BB2  
  
BB2:  
    if x > 0 goto BB3 else goto BB4  
  
BB3:  
    sum += x  
    --x  
    goto BB2  
  
BB4:  
    use(sum)  
    ...
```

BB_2 是 loop header，每次迭代前都檢查條件。 $3 \rightarrow 2$ 讓 CFG 形成 cycle； $2 \rightarrow 4$ 則是離開迴圈的 exit edge。

4. do / while

```
do {  
    read_next();  
} while (!ok);  
use_data();
```

Lowering 後：

BB1:

```
    read_next()  
    goto BB2
```

BB2:

```
    if !ok goto BB1 else goto BB3
```

BB3:

```
    use_data()  
    ...
```

條件放在 body 後方，所以 BB_1 至少會執行一次；這就是它和 while CFG 的主要差異。

5. for、break 與 continue

```
for (int i = 0; i < n; ++i) {  
    if (a[i] < 0) continue;  
    if (a[i] == 0) break;  
    sum += a[i];  
}  
use(sum);
```

for (init; cond; step) body 可以拆成 init、condition、body、step 與 exit：

```
BB1: i = 0; goto BB2  
BB2: if i < n      goto BB3 else goto BB7  
BB3: if a[i] < 0   goto BB6 else goto BB4  
BB4: if a[i] == 0  goto BB7 else goto BB5  
BB5: sum += a[i]; goto BB6  
BB6: ++i; goto BB2  
BB7: use(sum); ...
```

continue 必須跳到 step 的 BB_6 ，不能直接跳回 condition，否則會漏掉 ++i。break 則直接跳到 loop exit BB_7 。

6. Short-circuit : && 與 ||

```
if (a != 0 && b / a > 2)
    hit();
next();
```

&& 只有在左側為 true 時才會計算右側，所以兩個判斷通常位於不同 BB：

```
BB1:
    if a != 0 goto BB2 else goto BB4

BB2:
    if b / a > 2 goto BB3 else goto BB4

BB3:
    hit()
    goto BB4

BB4:
    next()
    ...
```

這張 CFG 也解釋了為什麼 $a == 0$ 時不會計算 b / a 。對 $a || b$ 而言則相反：左側為 true 時直接走 true edge，只有左側為 false 才計算右側。

7. switch

```
switch (op) {  
case 0: y = 10; break;  
case 1: y = 20; break;  
default: y = -1;  
}  
use(y);
```

最單純的 lowering 是一串比較：

```
BB1: if op == 0 goto BB3 else goto BB2  
BB2: if op == 1 goto BB4 else goto BB5  
BB3: y = 10; goto BB6  
BB4: y = 20; goto BB6  
BB5: y = -1; goto BB6  
BB6: use(y); ...
```

真正的編譯器也可能保留多路 switch terminator，或產生 jump table。無論機器碼如何實作，CFG 的 dispatch block 都有多個可能 successors，各個 case 最後再連到 join point。

8. 提早 return 每個 return 都會立刻結束目前 BB，而且在函式內沒有 successor。若某個分析希望只有一個出口，可以先將回傳值存入暫存變數，再全部跳到共用出口：

```
BB_return_negative:  
    result = -1  
    goto BB_exit  
  
BB_return_answer:  
    result = answer  
    goto BB_exit  
  
BB_exit:  
    return result
```

這是 compiler pass 常見的 CFG normalization，但不是所有 compiler 都必須這樣做。

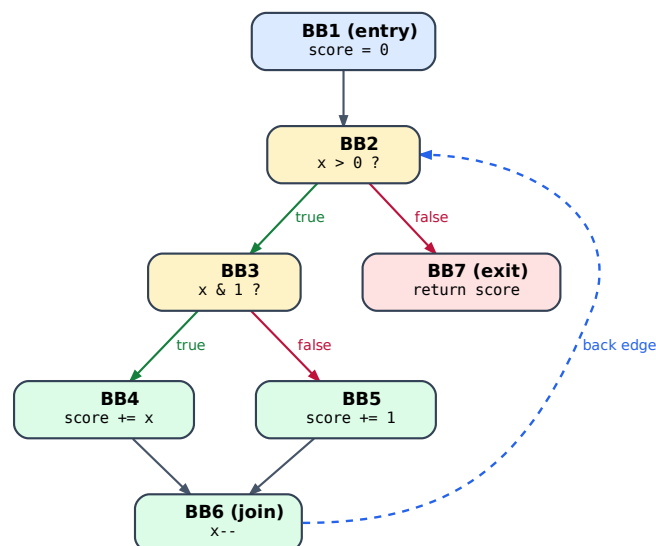
完整範例：從 C、goto-style 到 CFG

題目開頭的 CFG 對應以下 C function：

```
int calc(int x) {
    int score = 0;
    while (x > 0) {
        if (x & 1)
            score += x;
        else
            score += 1;
        --x;
    }
    return score;
}
```

先 lower 成只含明確 jump 的 pseudo IR：

```
BB1: score = 0; goto BB2
BB2: if x > 0 goto BB3 else goto BB7
BB3: if x & 1 goto BB4 else goto BB5
BB4: score += x; goto BB6
BB5: score += 1; goto BB6
BB6: --x; goto BB2
BB7: return score
```



上述 goto-style pseudo IR 建立出的 CFG。

BB_4 與 BB_5 不能合併，因為兩者是不同 branch targets。 BB_6 也必須從 join point 開始，因為它同時有 BB_4 、 BB_5 兩個 predecessors。最後， $BB_6 \rightarrow BB_2$ 回到 loop header，形成迴圈。

支配關係 (Dominance)

在具有入口的 CFG 中，若從 entry 到 v 的**每一條路徑**都一定經過 u ，就稱 u **支配** v ，記為 $u \text{ dom } v$ 。

- 每個節點都支配自己；
- 若 $u \neq v$ 且 u 支配 v ，稱為 strict dominance；
- v 的 **immediate dominator**，記為 $\text{idom}(v)$ ，是最靠近 v 的 strict dominator。

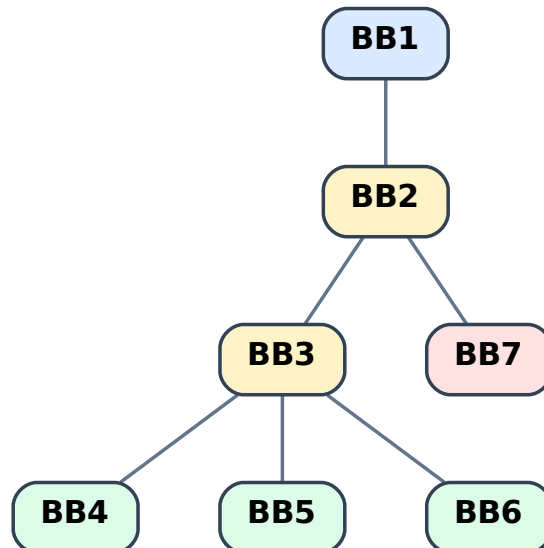
以前面的七個 BB 為例：

節點	Dominators	idom
BB_1	$\{BB_1\}$	無
BB_2	$\{BB_1, BB_2\}$	BB_1
BB_3	$\{BB_1, BB_2, BB_3\}$	BB_2
BB_4	$\{BB_1, BB_2, BB_3, BB_4\}$	BB_3
BB_5	$\{BB_1, BB_2, BB_3, BB_5\}$	BB_3
BB_6	$\{BB_1, BB_2, BB_3, BB_6\}$	BB_3
BB_7	$\{BB_1, BB_2, BB_7\}$	BB_2

BB_4 不支配 BB_6 ，因為可以沿著 $BB_3 \rightarrow BB_5 \rightarrow BB_6$ 到達 BB_6 而不經過 BB_4 。同理， BB_5 也不支配 BB_6 ；但不論走哪一臂都一定先經過 BB_3 ，所以 $\text{idom}(BB_6) = BB_3$ 。

Dominator Tree

把每個非 entry 節點 v 接到 $\text{idom}(v)$ ，就得到 dominator tree：



完整範例的 dominator tree。父節點是子節點的 immediate dominator。

在 dominator tree 中， u 支配 v ，恰好等價於 u 是 v 的 ancestor。注意 dominator-tree edges 代表的是「必經關係」，不一定是原 CFG 中的 control-flow edges。

Dominance 是許多 compiler analyses 的基礎，例如：

- 確認 SSA value 的 definition 是否支配它的 uses；
- 配合 dominance frontier 決定需要放置 phi 的 join points；
- 判斷某段運算能否安全移動；
- 找出 loop header 與 natural loop。

Dominance 與 Loop

對 CFG edge $u \rightarrow h$ ，若 h 支配 u ，則這條邊稱為 **back edge**， h 通常是 loop header。完整範例中， BB_2 支配 BB_6 ，所以 $BB_6 \rightarrow BB_2$ 是 back edge。

這條 back edge 對應的 natural loop 為：

{ BB_2 , BB_3 , BB_4 , BB_5 , BB_6 }

BB_2 是 header， BB_6 是跳回 header 的 latch， $BB_2 \rightarrow BB_7$ 是離開迴圈的 exit edge。
 BB_7 不在 loop 內。

一般有向圖中的 cycle 不一定都具有單一、支配整個 cycle 的 header；這類 CFG 稱為 irreducible。由 while、for 等結構化語法產生的迴圈，通常會形成較容易分析的 reducible CFG。

回到本題

本題已經直接給出 CFG，不需要自行把 C code lowering 成 goto-style。輸入的每個點就是一個 BB，每條有向邊就是一種可能的控制轉移；一次程式執行，就是從 entry 1 沿 CFG 走到 exit N 。性能計數器所測量的，正是這些 control-flow edges 實際被通過的次數。

TODO 競賽名稱
TODO 競賽名稱 試題本

附錄

TODO